INF-253 Lenguajes de Programación
Tarea 3 2026-1: La Amenaza de Sephiroth

Profesores: José Luis Martí Lara, Wladimir Ormazábal Orellana
Ayudante coordinador: Bastián Ríos Lastarria
Agradecimientos especiales a Carlos Bravo por la idea base de esta tarea.

20 de abril de 2026

Índice

1. Contexto **2**

2. Descripción de la tarea **2**
2.1. El Bucle de Juego e Idea Principal 2

3. Zonas del Mundo (Paquete `mapa`) **2**
3.1. Sector 7 (Zona Segura y Base de Operaciones) 2
3.2. Gongaga (Zona salvaje) 3
3.3. Núcleo del Planeta (Jefe Final) 3

4. Entidades y Combate (Paquete `entidades`) **3**
4.1. El Jugador y su Arma (Clases Anidadas) 3
4.2. Enemigos y Polimorfismo 4
4.3. Catálogo de Enemigos y Estadísticas 4

5. Fórmulas de Sistema y Componentes (Paquete `componentes`) **5**
5.1. Fórmulas de Combate y Gestión de Recursos 5
5.2. Progresión: Experiencia y Niveles 6
5.3. Economía de la Tienda 6

6. Reglas de Victoria, Derrota y Persistencia **6**
6.1. Derrota (Muerte de Cloud en Zonas de Peligro) 6
6.2. Victoria (Fin del Juego) 6

7. Diagrama de clases, atributos y métodos mínimos **7**

8. Sobre la entrega **8**

9. Calificación **9**
9.1. Entrega 9
9.1.1. Entrega Mínima (total 30 pts) 9
9.2. Descuentos 10

1. Contexto

El mundo está en peligro. Sephiroth, un antiguo héroe caído, se ha atrincherado en las profundidades del planeta y está reuniendo energía para lanzar un hechizo devastador que lo destruirá todo. Tú controlas a **Cloud**, un mercenario que debe prepararse para hacerle frente. Para lograrlo, Cloud deberá viajar entre distintas zonas, enfrentar enemigos, encontrar "Materias" (esferas mágicas) para fortalecerse y gestionar sabiamente sus estadísticas como Fuerza (**ATK**) Salud (**HP**) y Magia (**MP**).

2. Descripción de la tarea

La tarea consistirá en construir un prototipo de juego de rol (RPG) por turnos y exploración en consola utilizando Java. El objetivo es aprender a aplicar conceptos de Programación Orientada a Objetos: herencia, polimorfismo, interfaces, composición, clases anidadas y enumeraciones con estado. Para mantener un diseño modular y escalable, el código debe estar organizado en tres paquetes obligatorios: `mapa`, `entidades` y `componentes`.

2.1. El Bucle de Juego e Idea Principal

El juego se divide en tres zonas y fases. Mediante un menú en consola, el jugador puede elegir viajar a otras zonas, gestionar su equipo o explorar la zona actual para desencadenar eventos.

Comienzo: Al comenzar, el jugador, controlando a **Cloud** aparece en **Sector 7**, la zona segura e inicial. El jugador comenzará con nivel 1 e irá progresando en esta zona inicial hasta conseguir el nivel necesario para entrar en la siguiente zona. **Sector 7** provee a Cloud con una zona de entrenamiento especializada donde Cloud puede ganar Experiencia de forma segura sin morir.

Combate: En combate, el jugador podrá elegir entre **atacar** físicamente, explotar debilidades usando **magia** elemental, **curarse** (si posee el elemento equipado) o **huir**. Si los Puntos de Salud de Cloud llegan a 0 en zonas de peligro, es rescatado y devuelto al Sector 7, pero **pierde toda su chatarra y todas las Materias que estén en su Mochila.** Las Materias que estén equipadas en el Arma están a salvo y no se pierden.

3. Zonas del Mundo (Paquete `mapa`)

Existen tres locaciones principales:

3.1. Sector 7 (Zona Segura y Base de Operaciones)

Aquí Cloud no corre peligro. Al explorar, el jugador accede a dos mecánicas de la zona:

- **Simulador de Combate (`iniciarSimulador`):** Permite a Cloud luchar contra un `EnemigoSimulador` holográfico para ganar experiencia de forma segura. Si Cloud pierde aquí, no hay penalización, simplemente el combate termina y Cloud queda con 1 HP.
- **Tienda de Chatarra (`abrirTienda`):** Permite gastar la "Chatarra" obtenida en combates salvajes para adquirir `Mejoras` a las estadísticas base de Cloud de forma permanente.

3.2. Gongaga (Zona salvaje)

Una jungla donde Cloud recolecta recursos. Al invocar el método de exploración, se ejecuta un evento aleatorio:

- Puede encontrar una `Materia` que se añade directamente a la mochila del jugador. La Materia encontrada debe instanciarse asignándole un `Elemento` aleatorio con igual probabilidad entre `FUEGO, HIELO, RAYO` o `CURA`. (Prob. 30 % de encontrar Materia)
- Puede sufrir una emboscada. Si ocurre, el método `generarGrupoEnemigo()` determinará si Cloud se enfrenta a 1, 2 o 3 `EnemigoSalvaje` simultáneamente. (Prob. 70 % de ser emboscado)
- Pueden aparecer hasta **3** entidades al iniciar combate por emboscada, con un **60 %** que aparezca 1, **30 %** que aparezcan 2 y un **10 %** que aparezcan 3 y pueden ser distintos.
- En caso de haber más de un enemigo la probabilidad de que ataquen juntos se reduce a un 50 % en caso de ser 2 y un 33 % en caso de ser 3, en caso contrario, solo atacará uno solo por turno.

3.3. Núcleo del Planeta (Jefe Final)

El centro del planeta. La dificultad de esta zona exige de manera estricta que Cloud sea al menos **Nivel 20** y tenga un mínimo de 2 materias equipadas en su Arma. Al explorar, inicia el combate directamente contra Sephiroth. El objetivo del juego es vencerlo y no se puede huir de este combate.

4. Entidades y Combate (Paquete `entidades`)

Este paquete contiene a todos los actores del juego. El sistema de combate debe aprovechar la herencia y el polimorfismo para determinar cómo cada entidad recibe o inflige daño.

4.1. El Jugador y su Arma (Clases Anidadas)

La clase `Jugador` gestiona el nivel, la experiencia, la chatarra y un inventario general ilimitado (`mochila`) donde se guardan las materias inactivas. Para canalizar la magia, Cloud cuenta con su espada. **La clase `Arma` debe ser una clase anidada dentro de `Jugador` de manera *obligatoria*.**

- Esta estructura permite que el Arma acceda directamente a las estadísticas de Cloud para calcular el daño.
- El Arma tiene un límite de **5 ranuras** para equipar Materias. Cloud solo puede lanzar magias de los elementos que tenga actualmente equipados.

Al iniciar el juego en Nivel 1, el componente Estadísticas de Cloud debe inicializarse con los siguientes valores base: **HP Máximo: 200, MP Máximo: 50, Fuerza: 15, Magia: 15**. Su `chatarra` y `expActual` comienzan en 0

4.2. Enemigos y Polimorfismo

Todos los monstruos heredan de la clase abstracta `Enemigo`, y tienen diversas propiedades que los diferencian. Los enemigos **siempre** atacarán después del ataque/acción de Cloud.

- `EnemigoSimulador`**:** Entidad de entrenamiento del Sector 7. No posee debilidades elementales. Su ataque está condicionado para nunca reducir el HP de Cloud por debajo de 1. pueden aparecer hasta **2** de estas entidades al iniciar combate. Probabilidad a definir por el estudiante.

- `EnemigoSalvaje`**:** Monstruos de Gongaga altamente sensibles al poder elemental. Cada uno posee distintas debilidades, resistencias e inmunidades.
  Al ser atacados por un **ataque mágico** de Cloud el método `evaluarDebilidad` de la interfaz `Vulnerable` retornará uno de los siguientes valores según el elemento utilizado, que se multiplicará al daño hecho por Cloud: 2,0 **(debilidad)**, 0,5 **(resistencia)**, 0,0 **(inmunidad)** o 1,0 (neutro).

- `Sephiroth`**:** Jefe Final. No posee debilidades elementales. Posee un `contadorSuperNova` que aumenta cada turno y actúa como un reloj para un ataque devastador. Si este contador llega a su límite de **10 turnos**, ejecuta `lanzarSuperNova()`, aniquilando a Cloud instantáneamente. El contador puede ser reiniciado a 0 por Cloud si éste logra atacarle con un ataque **Límite**.

4.3. Catálogo de Enemigos y Estadísticas

Para instanciar los enemigos en las distintas zonas, los constructores deben inicializar los siguientes valores base de HP, Fuerza (para su método `atacar`) y sus afinidades elementales:

- **Soldado común** (Enemigo de simulación):
  - **HP:** 50 | **Fuerza:** 15
- **Planta Carnívora** (Enemigo Salvaje):
  - **HP:** 80 | **Fuerza:** 15
  - **Debilidades:** `FUEGO` y `HIELO`.
  - **Inmunidades:** `RAYO`.
- **Sapo de la Jungla** (Enemigo Salvaje):
  - **HP:** 60 | **Fuerza:** 12
  - **Debilidades:** `RAYO` y `HIELO`.
  - **Resistencias:** `FUEGO`.
- **Robot Centinela** (Enemigo Salvaje): Máquina oxidada pero muy resistente.
  - **HP:** 100 | **Fuerza:** 20
  - **Debilidades:** `RAYO`.
  - **Resistencias:** `FISICO` y `HIELO`.
- **Sephiroth** (Jefe Final): El enemigo definitivo a vencer en el Núcleo del Planeta.

- **HP:** 500 | **Fuerza:** 40
- **Afinidades:** No posee debilidades (multiplicador 1,0 a todo).

Nota: Al igual que Cloud, el daño físico de los ataques básicos de los enemigos se calcula aplicando la fórmula $\lfloor Fuerza \times 1{,}25 \rfloor$*. Los enemigos no consumen MP al atacar.*
Todos los enemigos ***tienen una probabilidad del 85 % de acertar su ataque*** *cada turno que pasa,* ***a excepción de Sephiroth****, el cual cuenta con una precisión del 90 %.*
El catálogo de enemigos son ***instancias*** *de las clases previamente definidas. Se les da libertad de hacer subclases de la clase ya asignada si así lo prefieren pero se recomienda mantener cada enemigo como una instancia de su clase.*

5. Fórmulas de Sistema y Componentes (Paquete `componentes`)

Este paquete guarda los bloques de construcción del juego. Cualquier clase o enumeración adicional que se quiera agregar a la tarea para mejorar la jugabilidad debe ir en este apartado.

5.1. Fórmulas de Combate y Gestión de Recursos

Todos los cálculos resultantes deben redondearse al entero más cercano o truncarse (casteo a `int`). Las magias dependen del *enum* `Elemento` (`FUEGO`, `HIELO`, `RAYO`, `CURA`, `FISICO`).

- **Ataque Físico:** Daño estándar sin coste de MP.

$$Daño_{fisico} = \lfloor Fuerza \times 1{,}25 \rfloor$$

- **Ataque Mágico:** El daño se potencia al equipar varias materias idénticas, pero también incrementa su consumo de MP. Si n es la cantidad de materias del elemento seleccionado en las ranuras del Arma:

$$Daño_{magico} = \lfloor Magia \times (1{,}0 + (0{,}5 \times n)) \rfloor$$

$$Costo_{MP} = 10 + (5 \times n)$$

  Nota: Tras calcular este daño base, se debe multiplicar por el valor retornado por el método ***`evaluarDebilidad`*** *del enemigo.*
  También notar que uno de los elementos mágicos es ***`CURA`****. Este elemento* ***cura a Cloud*** *siguiendo la misma fórmula de daño al enemigo pero como efecto positivo hacia el Jugador. La curación no puede sobrepasar el límite de vida máxima que tenga el Jugador.*

- **Ataque Límite:** Un ataque físico devastador que ignora el consumo de MP. Para utilizarlo, Cloud posee una Barra de Límite (de 0 a 100) que se carga exclusivamente durante los combates:

  - Al recibir daño: $Carga \mathrel{+}= \lfloor \frac{Daño_{recibido}}{2} \rfloor$
  - Al infligir daño: $Carga \mathrel{+}= \lfloor \frac{Daño_{realizado}}{5} \rfloor$

  Al llegar a 100, la acción se habilita en el menú. El daño ignora los multiplicadores de debilidad y se calcula como:

$$Daño_{limite} = Fuerza \times 5$$

  Nota: Tras ejecutarse, la barra de límite vuelve a 0 de forma obligatoria.

5.2. Progresión: Experiencia y Niveles

Cloud comienza en Nivel 1. La experiencia requerida para avanzar de nivel sigue una curva lineal:

$$XP_{necesaria} = 10 \times Nivel_{actual}$$

Al subir de nivel, las estadísticas base del componente `Estadisticas` de Cloud aumentan automáticamente: **+10 HP Máximo, +5 MP Máximo, +4 Fuerza y +6 Magia**.
Las recompensas por combate varían según el rival derrotado (calculadas al instanciarse el enemigo):

- `EnemigoSimulador`**:** Otorga un valor aleatorio entre **15 y 20 XP**. No suelta botín.
- `EnemigoSalvaje`**:** Otorga un valor aleatorio entre **80 y 100 XP**, y suelta entre **50 y 75 de Chatarra**.

5.3. Economía de la Tienda

La chatarra obtenida en Gongaga se utiliza en el Sector 7 para comprar instancias de la clase `Mejora`, las cuales aplican aumentos permanentes (regidos por el *enum* `TipoStat`):

- **Mejora de Vitalidad (+20 HP Máximo):** Costo 100 Chatarra.
- **Mejora de Éter (+10 MP Máximo):** Costo 120 Chatarra.
- **Mejora Física (+10 Fuerza):** Costo 150 Chatarra.

Al comprar una `Mejora`, sus efectos se aplican de manera inmediata y permanente a las `Estadisticas` base del Jugador. No ocupan espacio en el inventario.

6. Reglas de Victoria, Derrota y Persistencia

6.1. Derrota (Muerte de Cloud en Zonas de Peligro)

Como se definió en las reglas generales, si los HP de Cloud llegan a 0 durante un combate en Gongaga o en el Núcleo del Planeta:

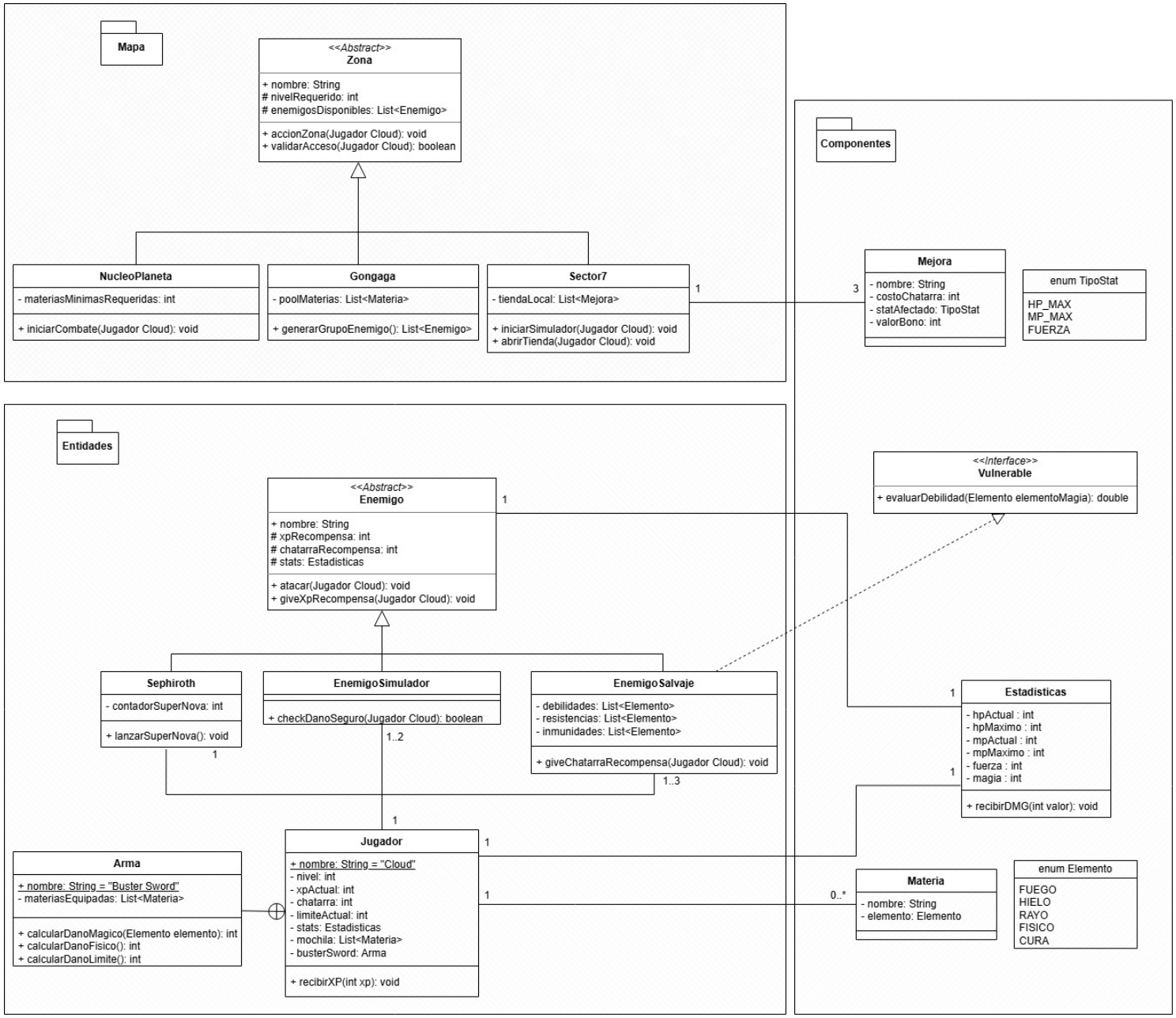
- El combate termina inmediatamente y Cloud es retornado al menú del **Sector 7** con su HP restaurado.
- La penalización se aplica vaciando sus reservas: su chatarra vuelve a 0 y su lista de mochila se vacía. Las materias equipadas en las ranuras del Arma quedan intactas.

6.2. Victoria (Fin del Juego)

El objetivo principal del prototipo es derrotar a Sephiroth. Cuando los HP de `Sephiroth` llegan a 0 o un valor negativo tras un ataque de Cloud:

- El combate se interrumpe de inmediato (el enemigo no puede realizar un turno de ataque si ya fue derrotado).
- Se debe imprimir por consola un mensaje de victoria anunciando que Cloud ha salvado el planeta, junto con cualquier estadística final o mensaje que el estudiante desee mostrar (Nivel final, turno en el que ganó, secuencia ASCII, etc.).
- El programa debe finalizar su ejecución de manera limpia y automática.

7. Diagrama de clases, atributos y métodos mínimos



Información a considerar: Lo único que se permite variación respecto al diagrama son los retornos y argumentos de los métodos entregados, los nombres y atributos deben respetarse tal cual se entregan.
Son libres de agregar cuantas clases, atributos y métodos adicionales quieran, siempre y cuando no sin uso a los elementos del diagrama.

8. Sobre la entrega

- El código debe venir indentado y ordenado.
- Se debe entregar los siguientes archivos:
  - Main.java
  - **Mapa**/
    - {Zona, NucleoPlaneta, Gongaga, sector7}.java
  - **Entidades**/
    - {Enemigo, Sephiroth, EnemigoSimulador, EnemigoSalvaje, Jugador}.java
  - **Componentes**/
    - {Mejora, TipoStat, Vulnerable, Estadisticas, Elemento, Materia}.java
  - README.txt / README.md
  - **makefile**
- Los métodos, a excepción de los getters y setters, deben ir comentados, explicando clara y brevemente lo que realiza, los parámetros que recibe y los que devuelve (en caso de que devuelva algo).

► **El Uso de IA está prohibido**. Cualquier sospecha de uso de herramientas de inteligencia artificial podrá conllevar descuentos en la evaluación o citación a defensa. En caso de comprobarse el uso de inteligencia artificial, la entrega será evaluada con nota 0 y notificada al profesor.)

- Si no existe orden en el código habrá descuento. **Se harán descuentos por funcion no comentada**
- La entrega debe realizarse en un archivo comprimido en tar.gz y debe llevar el nombre: Tarea3LP_RolAlumno.tar.gz.
  Ej.: Tarea3LP_202573000-k.tar.gz.
- El archivo README.txt debe contener el nombre y rol del alumno e instrucciones detalladas para la correcta utilización de su programa.
- La entrega será vía aula y el plazo máximo de entrega es hasta el día **12 de Mayo, 23:59 horas, vía aula.**
- Por cada día de atraso se descontarán 20 puntos (10 puntos dentro de la primera hora)
- Las copias y Uso de código generado por IA serán evaluadas con nota 0 y se informarán a las respectivas autoridades.
- Solo se pueden realizar consultas respecto a la tarea hasta 2 días antes de la entrega.
- Las consultas del enunciado se debe realizarse mediante el foro de la tarea disponible en AULA.

9. Calificación

9.1. Entrega

Para la calificación de su tarea, debe realizar una entrega con requerimientos mínimos que otorgarán 30 pts base, luego se entregará puntaje dependiendo de los otros requerimientos que llegue a cumplir.

9.1.1. Entrega Mínima (total 30 pts)

1. **Instanciación y Composición:** Al ejecutar, el juego inicia con el menú en el **Sector 7**. `Cloud` debe estar instanciado correctamente con sus `Estadisticas` y poseer su clase anidada `Arma`. (10 pts)

2. **Bucle de Combate Básico:** Debe ser posible ingresar al **Simulador de combate**, iniciar un combate contra al menos 1 `EnemigoSimulador`, y poder ejecutar al menos un turno completo de ataque físico, reduciendo el HP del enemigo. (20 pts)

- **Flujo, Exploración y RNG (20 pts)**
  - **(5 pts) Navegación correcta por el menú entre Zonas**, validando correctamente que no se pueda entrar al Núcleo sin ser Nivel 20 o a Gongaga sin Nivel 5.
  - **(10 pts) Lógica de Emboscada en Gongaga (70 %)**: Generación aleatoria de 1 a 3 enemigos con sus respectivas probabilidades de ataque conjunto (50 % o 33 %).
  - **(5 pts) Lógica de Botín en Gongaga (30 %)**: Generación de Materia con elemento aleatorio y guardado en la mochila de Cloud.
- **Sistema de Combate y Magia (25 pts)**
  - **(10 pts)** Implementación de Magia Ofensiva: Cálculo de daño escalado correctamente según cantidad de materias y deducción del coste de MP.
  - **(7 pts)** Multiplicadores Elementales: El daño mágico se multiplica de forma correcta (x2.0, x0.5, etc.) interactuando con la interfaz `Vulnerable` de los enemigos.
  - **(5 pts)** Magia CURA funcional: Restaura salud utilizando la fórmula de daño, con restricción estricta de no superar el HP Máximo de Cloud.
  - **(3 pts)** Mecánica de Huida: Implementación de la probabilidad de éxito al huir (50 %); si falla, Cloud pierde el turno.
- **Mecánicas de Jefe Final y Límite (15 pts)**
  - **(5 pts)** Barra de Límite: Se carga correctamente según el daño recibido y realizado. Al llegar a 100, permite ejecutar el ataque y se vacía a 0.
  - **(5 pts)** Reloj de Sephiroth: El `contadorSuperNova` incrementa cada turno. Si llega a 10, Cloud es aniquilado.
  - **(5 pts)** Interrupción de Jefe y Reinicio: El ataque Límite de Cloud reinicia el contador de Sephiroth a 0. Si Cloud muere, la instancia de Sephiroth se restaura por completo para el próximo intento.
- **Progresión y Economía (10 pts)**

- **(5 pts)** Subida de Nivel: Cloud escala su experiencia de forma lineal y, al subir de nivel, aumentan automáticamente sus 4 estadísticas base.
- **(5 pts)** Tienda Funcional: El sistema descuenta correctamente la chatarra y aplica los bonos de las `Mejoras` permanentemente al componente `Estadisticas` de Cloud.

9.2. Descuentos

- Falta de Makefile o no funcional (-100 pts)
- Falta de clases y/o paquetes del diagrama (-100 pts)
- Código no compila (-100 pts)
- Falta de atributos y/o métodos mínimos del diagrama (-20 pts c/u)
- Falta de README (–20 pts)
- Falta de orden (Max -20 pts)
- interfaz confusa, poco clara o inestable (máx -30 pts)
- Atributo público que no corresponde (cada uno, máx -30, -5 pts c/u)
- Warnings de compilación (-5 pts c/u)
- Falta de comentarios sobre cada método (–5 pts c/u, máx. –30 pts). (a excepción de getters y setters)
- Uso de librerías **no estándar** de C. (-100 pts)
- Compilación Unificada (-5 pts)
- Warnings (-5 pts c/u)
- Falta de Makefile (-100 pts)
- Falta de README (–20 pts)
- Falta de información obligatoria en el README (nombre, rol, instrucciones) (–5 pts c/u)
- Falta de orden o mala indentación en el código (–5 a –20 pts según gravedad)
- Mal nombre en los archivos entregados (–5 pts c/u; –10 pts si es el `.tar.gz`)
- Uso de código y/o texto generado por IA implicará un 0 y se le informará a las respectivas autoridades.
- Entrega tardía (–10 pts si es dentro de la primera hora; –20 pts por cada día o fracción de retraso)
- En caso de existir nota negativa, esta será remplazada por un 0.